

RITESH VISHWAKARMA

MERN Stack Interview Preparation Sheet

Full Stack Developer Intern · Digi-Lateral Solutions Interview

JavaScript · React.js · Node.js · MongoDB · REST APIs · Projects

#	Chapter	Topic
01	Self Introduction	HR Round — What to say & how
02	HR Questions	Why hire you, strengths, goals
03	JavaScript Core	Closures, async, event loop, ES6+
04	React Fundamentals	Hooks, lifecycle, patterns
05	Backend & Node.js	Express, middleware, REST APIs
06	MongoDB & Mongoose	Schema, queries, relationships
07	Auth Systems	JWT, OAuth2, cookies, security
08	Your Projects	ONYX, Perplexity, PlayPulse, BharOS
09	Coding Round	Blank screen strategy + 8 patterns
10	Day-Before Checklist	Final 24-hour plan

01 SELF INTRODUCTION

Script — Exactly Yah Bolo

"Hi, I'm Ritesh. I completed my B.Com and transitioned into full-stack development. I work with React and the MERN stack. Recently I built ONYX — a fashion e-commerce platform with role-based authentication, cart management, and a seller dashboard. I've also built an AI-powered search engine and a game discovery platform. I enjoy understanding how systems work end-to-end — not just writing code, but knowing why each piece exists. I'm looking for an opportunity to contribute to real projects while continuing to grow."

Why This Works

- Opens with who you are — no fluff
 - "Transitioned" not "I am learning" — frames you as a developer, not a student
 - ONYX is named immediately — concrete proof, not vague claims
 - Ends with what you want — clear, not desperate
 - Time: exactly 45-60 seconds — not too short, not rambling
-

What NOT to Say

- Never say: "I am learning MERN stack" — sounds unfinished
- Never say: "I am a fresher" — they know, no need to highlight
- Never say: "I don't have experience but..." — starts with weakness
- Never start with your college or degree — ends with it instead

02 HR QUESTIONS

Q: Why should we hire you?

I may not know everything, but I learn by building real things, not watching tutorials. ONYX has auth, role-based flows, cart logic, and a seller dashboard — I built all of it. Perplexity City had LangChain breaking changes mid-build — I debugged and fixed it. I'm comfortable being stuck, because I know how to get unstuck.

♦ *End with proof, not promise. "I built X" beats "I will do Y".*

Q: What is your biggest weakness?

I sometimes spend too long debugging before asking for help. I've been working on this — now I set a 30-minute rule: if I can't find the root cause in 30 min, I document what I've tried and ask. It's made me faster without making me dependent.

♦ *Real weakness + what you did about it = strong answer.*

Q: Where do you see yourself in 2 years?

I want to be a developer who can take a feature from requirements to deployment independently — thinking about architecture, writing the code, and handling production issues. This internship is a step toward that.

♦ *Ambitious but grounded. Not "I want to be CTO".*

Q: Why this company / this role?

I want to work on real products with a real team. Reading code, debugging together, and shipping features in a team context teaches things no solo project can. A MERN internship is exactly the environment where I can grow fastest right now.

Q: Are you comfortable with a stipend of Rs 5000?

Yes. My priority right now is learning in a real environment — that's worth more than a high stipend at a place where I wouldn't grow.

03 JAVASCRIPT CORE

Q: var vs let vs const — what is the actual difference?

var is function-scoped and hoisted with value undefined. let and const are block-scoped and hoisted but NOT initialized — accessing them before declaration gives ReferenceError (this zone is called Temporal Dead Zone). const cannot be reassigned but the object it points to can be mutated.

```
var x = 1; // function scope, hoisted let y = 2; // block scope, TDZ before declaration
const z = 3; // block scope, cannot reassign const obj = {a:1}; obj.a = 2; // allowed – mutation, not reassignment
```

♦ In ONYX, all variables are let/const. Mention this if asked.

Q: What is a closure? Give a real example.

A closure is when a function remembers the variables from its outer scope even after that outer function has returned. The inner function "closes over" those variables.

```
function makeCounter() { let count = 0; // outer variable return function() { count++; // inner function remembers count return count; }; } const counter = makeCounter();
counter(); // 1 counter(); // 2 – count persists!
```

♦ Real use: React's useState hook internally uses closures to preserve state.

Q: What is the event loop? How does JavaScript handle async code?

JavaScript is single-threaded — one thing at a time. But it can handle async operations through the event loop. When async code (setTimeout, fetch) is called, it goes to the Web API, runs there, and the callback is placed in the task queue when done. The event loop checks: is the call stack empty? If yes, it picks from the queue.

```
console.log("1"); // Call stack setTimeout(() => { // Goes to Web API console.log("2"); }, 0); console.log("3"); // Call stack // Output: 1, 3, 2 // Even with 0ms delay – event loop waits for stack to clear
```

♦ Microtasks (Promises) run before macrotasks (setTimeout). Important detail.

Q: Promise vs async/await — what is the difference?

Both handle async operations. Promise uses .then().catch() chaining. async/await is syntactic sugar over Promises — same behavior, cleaner syntax. await pauses execution inside an async function until the Promise resolves.

```
// Promise style fetch("/api/user") .then(res => res.json()) .then(data => console.log(data)) .catch(err => console.error(err)); // async/await style (same thing, cleaner)
async function getUser() { try { const res = await fetch("/api/user"); const data = await res.json(); console.log(data); } catch(err) { console.error(err); } }
```

♦ In ONYX, all API calls use async/await. Mention this.

Q: What is hoisting?

JavaScript moves variable and function declarations to the top of their scope before execution. var declarations are hoisted and initialized as undefined. Function declarations are fully hoisted. let/const are hoisted but NOT initialized (TDZ).

```
console.log(x); // undefined (not error) – var hoisted var x = 5; greet(); // "Hello" – function fully hoisted
function greet() { console.log("Hello"); } console.log(y); // ReferenceError – let in TDZ let y = 10;
```

Q: map, filter, reduce — explain with examples.

map: transforms each element, returns new array of same length. filter: keeps elements that pass a condition, returns subset. reduce: accumulates values into a single result.

```
const nums = [1, 2, 3, 4, 5]; // map – double each nums.map(n => n * 2); // [2,4,6,8,10]
// filter – only evens nums.filter(n => n % 2 === 0); // [2,4] // reduce – sum all
nums.reduce((acc, n) => acc + n, 0); // 15 // Real use in ONYX: cart total
cart.reduce((total, item) => total + item.price * item.qty, 0)
```

♦ *Always connect examples to your projects.*

Q: What is destructuring? What is spread/rest?

Destructuring: extract values from objects/arrays into variables. Spread (...): expand iterable into individual elements. Rest (...): collect remaining elements into array.

```
// Destructuring const { name, role } = user; // object const [first, second] = arr; //
array // Spread const newArr = [...arr1, ...arr2]; const newObj = { ...user, role: "admin"
}; // Rest function sum(...nums) { return nums.reduce((a, b) => a + b, 0); }
```

Q: What is "this" in JavaScript?

this refers to the object that is calling the function. Its value depends on HOW the function is called, not where it is defined. In arrow functions, this is inherited from the surrounding scope (lexical this).

```
const obj = { name: "Ritesh", greet() { console.log(this.name); } // "Ritesh" }; // Arrow
function – no own "this" const greet = () => console.log(this); // window/undefined // In
React class components – this is the component instance // Arrow functions in React solve
"this" binding issues
```

04 REACT FUNDAMENTALS

Q: What is React? Why use it?

React is a JavaScript library for building user interfaces using reusable components. Instead of manually updating the DOM, React maintains a Virtual DOM — a lightweight copy. When state changes, React compares old and new Virtual DOM (diffing), and updates only what actually changed in the real DOM. This is called reconciliation.

◆ *Virtual DOM = performance. Components = reusability. Unidirectional data flow = predictability.*

Q: What is useState? How does it work?

useState is a Hook that gives a component its own memory. It returns the current value and a setter function. When the setter is called, React re-renders the component with the new value. State updates are asynchronous — you cannot rely on the updated value immediately after calling the setter.

```
const [count, setCount] = useState(0); // Update setCount(count + 1); // Functional update
(safe when depending on prev value) setCount(prev => prev + 1); // ONYX example
const [cartItems, setCartItems] = useState([]); const [isLoading, setIsLoading] =
useState(false);
```

◆ *In ONYX: cart state, auth state, loading states — all useState.*

Q: What is useEffect? When does it run?

useEffect handles side effects — things that happen outside React's rendering cycle: API calls, subscriptions, timers, DOM manipulation. It runs AFTER the component renders.

```
// Runs after every render useEffect(() => { ... }); // Runs only once (on mount)
useEffect(() => { fetchData(); }, []); // Runs when "id" changes useEffect(() => {
fetchProduct(id); }, [id]); // Cleanup (on unmount) useEffect(() => { const timer =
setInterval(fn, 1000); return () => clearInterval(timer); // cleanup }, []);
```

◆ *PlayPulse: I moved data fetching OUT of useEffect into Router loaders — mention this as an architectural decision.*

Q: useMemo vs useCallback — what is the difference?

Both are performance optimizations. useMemo memoizes a COMPUTED VALUE — prevents re-calculation on every render. useCallback memoizes a FUNCTION — prevents re-creation on every render. Use them only when there is a real performance problem, not by default.

```
// useMemo – expensive calculation const total = useMemo(() => { return
cartItems.reduce((sum, item) => sum + item.price, 0); }, [cartItems]); // useCallback –
stable function reference for child component const handleDelete = useCallback((id) => {
setItems(prev => prev.filter(item => item.id !== id)); }, []); // empty deps = never
recreated
```

◆ *Interviewer trap: "When do you NOT use useMemo?" Answer: when the calculation is cheap. Memoization itself has a cost.*

Q: What is a Custom Hook? When do you create one?

A Custom Hook is a JavaScript function that starts with "use" and calls other hooks. Create one when you find the same stateful logic repeated in multiple components.

```
// Without custom hook – repeated in 3 components // With custom hook: function useAuth()
{ const [user, setUser] = useState(null); const [isLoading, setIsLoading] =
useState(true); useEffect(() => { const token = localStorage.getItem("token"); if (token)
verifyToken(token).then(setUser); setIsLoading(false); }, []); return { user, isLoading };
} // Usage: const { user, isLoading } = useAuth();
```

◆ *ONYX uses useAuth() for auth state across buyer and seller components.*

Q: What is React Router? What are loaders?

React Router v6+ handles client-side navigation — different components render for different URLs without page reload. Data loaders (v6.4+) allow fetching data at the route level before the component renders, eliminating `useEffect` for data fetching.

```
// Traditional (useEffect in component) function GameList() { const [games, setGames] =
useState([]); useEffect(() => { fetchGames().then(setGames); }, []); } // With Router
Loader (PlayPulse approach) const router = createBrowserRouter([ { path: "/games", element:
, loader: async () => fetchGames(), // runs before render } ]); function GameList() { const
games = useLoaderData(); // data already here }
```

◆ *This is your architectural advantage in PlayPulse. Explain WHY — no loading state chaos, no stale data.*

Q: Controlled vs Uncontrolled Components?

Controlled: input value is controlled by React state. Every keystroke updates state. Uncontrolled: input manages its own value via DOM, accessed with `ref` when needed.

```
// Controlled (ONYX login form) const [email, setEmail] = useState("");
setEmail(e.target.value)} /> // Uncontrolled const emailRef = useRef(); // Access:
emailRef.current.value on submit
```

◆ *ONYX uses controlled components everywhere — easier validation, easier testing.*

Q: Why are Keys important in React lists?

Keys help React identify which items have changed, been added, or removed. Without keys, React re-renders the entire list on any change. With unique keys, it only updates the changed items.

```
// Wrong – index as key (breaks on reorder/delete) items.map((item, index) => ) // Correct
– unique stable ID items.map(item => )
```

◆ *In ONYX, `product._id` from MongoDB is used as key. Always use database ID.*

05 BACKEND — NODE.JS & EXPRESS

Q: What is Node.js? Why use it for backend?

Node.js is a JavaScript runtime built on Chrome's V8 engine. It allows running JavaScript outside the browser. Non-blocking I/O means it can handle many simultaneous connections without creating a new thread for each — efficient for API servers.

◆ *Key advantage: same language (JS) on frontend and backend — no context switching.*

Q: What is middleware in Express?

Middleware is a function that runs between the request and the response. It has access to req, res, and the next() function. Middleware can modify the request, send a response, or pass control to the next middleware by calling next().

```
// Middleware signature function authMiddleware(req, res, next) { const token = req.cookies.token; if (!token) return res.status(401).json({ error: "Unauthorized" }); const decoded = jwt.verify(token, process.env.JWT_SECRET); req.user = decoded; // attach user to request next(); // pass to next middleware/handler } // ONYX: every protected route runs this first router.get("/cart", authMiddleware, cartController.getCart);
```

◆ *ONYX backend: request → authMiddleware → roleMiddleware → controller. Three layers.*

Q: What are REST API HTTP methods and status codes?

REST uses HTTP methods to indicate intent. Status codes indicate result.

```
GET /products → 200 (list all) GET /products/:id → 200 or 404 (one item) POST /products → 201 (created) PUT /products/:id → 200 (updated) DELETE /products/:id → 200 or 204 (deleted) Status codes: 200 OK | 201 Created | 400 Bad Request 401 Unauthorized | 403 Forbidden | 404 Not Found 500 Internal Server Error
```

◆ *In ONYX, every route returns the correct status code. Mention this precision.*

Q: What is MVC architecture?

Model-View-Controller is a pattern to separate concerns. Model = data layer (Mongoose schemas). View = UI (React frontend). Controller = business logic (Express route handlers).

```
// ONYX backend structure routes/ → URL definitions, attach controllers controllers/ → handle request, call services, send response services/ → business logic (not in controller) models/ → Mongoose schemas // Request flow: // Route → Middleware → Controller → Service → Model → Response
```

◆ *This is exactly how ONYX is structured. Say: "I built ONYX using MVC — routes, controllers, services, and models are completely separated."*

06 MONGODB & MONGOOSE

Q: What is MongoDB? How is it different from SQL?

MongoDB is a NoSQL document database. Data is stored as JSON-like documents in collections (not rows in tables). No fixed schema — each document can have different fields. SQL has rigid schemas, joins, and ACID transactions by default. MongoDB is flexible and scales horizontally more easily.

◆ *ONYX uses MongoDB Atlas (cloud). Mention this shows production awareness.*

Q: What is Mongoose? What is a Schema?

Mongoose is an ODM (Object Document Mapper) for MongoDB. It lets you define Schemas — blueprints for documents — with types, validation, and defaults.

```
const productSchema = new Schema({ name: { type: String, required: true }, price: { type: Number, min: 0 }, stock: { type: Number, default: 0 }, seller: { type: ObjectId, ref: "User" }, // reference images: [String], createdAt:{ type: Date, default: Date.now } });
const Product = mongoose.model("Product", productSchema);
```

◆ *ONYX has User, Product, Cart, Order schemas with proper references.*

Q: What is populate() in Mongoose?

populate() replaces a referenced ObjectId with the actual document data from another collection. It's like a JOIN in SQL.

```
// Without populate – returns ObjectId only
const product = await Product.findById(id);
product.seller = ObjectId("64abc...")
// With populate – returns full seller document
const product = await Product.findById(id).populate("seller", "name email");
product.seller = { name: "Ritesh", email: "..." }
```

◆ *ONYX: when fetching cart, we populate product details inside each cart item.*

Q: Common Mongoose queries?

These are the most frequently asked and used:

```
// Find all
await Product.find();
await Product.find({ category: "fashion" });
// Find one
await Product.findById(id);
await Product.findOne({ email });
// Create
await Product.create({ name, price, seller: userId });
// Update
await Product.findByIdAndUpdate(id, { price: 999 }, { new: true });
// Delete
await Product.findByIdAndDelete(id);
// Increment stock
await Product.findByIdAndUpdate(id, { $inc: { stock: -1 } });
```

07 AUTH — JWT, OAUTH2, COOKIES

Q: What is JWT? How does it work?

JSON Web Token is a way to securely transmit information between parties as a signed JSON object. Structure: Header.Payload.Signature (three parts, base64 encoded, dot-separated). Server signs the token with a secret key. Client sends token on every request. Server verifies signature — no database lookup needed.

```
// Login – create token const token = jwt.sign( { id: user._id, role: user.role },
process.env.JWT_SECRET, { expiresIn: "7d" } ); res.cookie("token", token, { httpOnly:
true, sameSite: "strict" }); // Middleware – verify token const decoded =
jwt.verify(token, process.env.JWT_SECRET); req.user = decoded; // { id, role }
```

◆ *ONYX: role is in the JWT payload. Middleware reads role to decide buyer vs seller access. This is role-based authorization.*

Q: Cookie vs localStorage for storing JWT — which is better and why?

HttpOnly Cookie is more secure. localStorage is accessible by JavaScript — vulnerable to XSS attacks. HttpOnly cookies cannot be accessed by JavaScript at all — even if attacker injects malicious script, they cannot steal the token. ONYX stores JWT in HttpOnly cookies for exactly this reason.

```
// Secure cookie settings res.cookie("token", token, { httpOnly: true, // JS cannot access
secure: true, // HTTPS only (production) sameSite: "strict" // prevents CSRF });
```

◆ *This is a signal that you think about security, not just functionality.*

Q: What is OAuth2? How did you use it?

OAuth2 is an authorization framework that lets users log in with a third-party account (like Google) without sharing their password with your app. Flow: User clicks Google Login → Redirect to Google → User approves → Google sends authorization code → Server exchanges code for access token + refresh token.

```
// Perplexity City – Gmail OAuth2 issue I solved: // Problem: access token expires every 1
hour // Solution: store refresh token, auto-renew access token async function
getValidToken() { if (isTokenExpired(accessToken)) { const { token } = await
oauth2Client.refreshAccessToken(); accessToken = token.access_token; } return
accessToken; }
```

◆ *This is a real production problem you solved. Most freshers have never hit this. Say this confidently.*

08 YOUR PROJECTS — EXACT EXPLANATIONS

ONYX — Luxury Fashion Marketplace (3 min)

WHAT: Full-stack role-based fashion marketplace. Buyers browse and purchase. Sellers list and manage products. Separate flows, one codebase.

TECH: React + Tailwind (frontend), Node.js + Express + MongoDB (backend), JWT in HttpOnly cookies for auth, Cloudinary for image uploads.

KEY DECISIONS I MADE:

- Role-based auth: JWT payload contains role field. Middleware reads it. Wrong role = 403.
- Stock validation on checkout: prevents overselling without needing a distributed lock at this scale.
- MVC structure: routes → controllers → services → models. Business logic never in route handlers.
- HttpOnly cookie for JWT: chose security over convenience of localStorage.
- Cloudinary pipeline: multipart upload → SDK → CDN URL stored in MongoDB.

CHALLENGE + HOW I SOLVED IT:

"The hardest part was handling stock validation during simultaneous checkout. Two users could add the last item to their cart at the same time. I solved it by validating stock at the checkout endpoint — not at add-to-cart — and returning an error if stock dropped to zero between add and purchase. It's not a perfect solution at massive scale, but it's correct for this application size."

Perplexity City — AI Search Engine (2 min)

WHAT: AI-powered search engine. User types query → LangChain agent decides whether to call Tavily web search or answer from LLM knowledge → Groq LLM generates response → Markdown rendered on frontend.

WHAT MAKES IT DIFFERENT:

- It's not just "query → LLM response". The agent decides WHEN to search the web.
- I hit LangChain v1 breaking changes mid-build — AgentExecutor was removed, all imports changed.
- Fixed it by reading raw LangChain source on GitHub — no Stack Overflow answer existed.
- OAuth2 refresh token problem: Gmail access token expires every hour. Built auto-refresh logic.

PlayPulse — Game Discovery Platform (1.5 min)

WHAT: Game discovery app using RAWG API. Search, filter, favourites.

KEY ARCHITECTURAL DECISION:

"Instead of fetching data in useEffect inside components, I moved all data fetching into React Router v7 loaders. This means components never handle loading states — the router does. Navigation never shows stale data. Search state lives in the URL, not component state — so the back button works correctly and search results are shareable."

BharOS — Browser Desktop Environment (1 min)

WHAT: macOS-inspired interactive desktop environment running entirely in the browser. No backend. Pure React and Browser APIs.

- Draggable + resizable windows using react-rnd — each window tracks its own position in local state
- Battery Status API for live battery display — zero external libraries
- Real-time clock with setInterval — updates every second
- Architecture: each desktop app is isolated with local state — no global state needed

09 CODING ROUND — BLANK SCREEN STRATEGY

The Rule — Always Do This First

When you get a coding problem, DO NOT start typing immediately. Do this:

```
1. Read the problem out loud and confirm: "So you want me to build a [X] that does [Y], right?" 2. Write the skeleton first – always. 3. Add state. 4. Add UI structure. 5. Add logic.
```

ALWAYS start with this skeleton — no matter what the problem is:

```
import { useState, useEffect } from "react"; function Component() { // 1. STATE const [value, setValue] = useState(...); // 2. EFFECTS / HANDLERS // 3. RENDER return ( /* UI here */ ); }
```

■ Interviewers judge your *PROCESS*, not just your output. Thinking out loud = bonus points.

8 Problems They Will Most Likely Ask

■ Counter

Increment, Decrement, Reset buttons with display

```
const [count, setCount] = useState(0); return ( {count} setCount(c => c+1)}>+ setCount(c => c-1)}>- setCount(0)}>Reset );
```

■ Todo List

Add, display, delete todos

```
const [todos, setTodos] = useState([]); const [input, setInput] = useState(""); const addTodo = () => { if (!input.trim()) return; setTodos(prev => [...prev, { id: Date.now(), text: input }]); setInput(""); }; const deleteTodo = (id) => setTodos(prev => prev.filter(t => t.id !== id));
```

■ Search Filter

Filter a list as user types

```
const [query, setQuery] = useState(""); const items = ["React", "Node", "MongoDB", "Express"]; const filtered = items.filter(item => item.toLowerCase().includes(query.toLowerCase())); return ( setQuery(e.target.value)} /> {filtered.map(item => {item})} );
```

■ Fetch & Display

Call an API and show results

```
const [data, setData] = useState([]); const [loading, setLoading] = useState(true); const [error, setError] = useState(null); useEffect(() => { fetch("https://jsonplaceholder.typicode.com/users") .then(r => r.json()) .then(d => { setData(d); setLoading(false); }) .catch(e => { setError(e.message); setLoading(false); }); }, []); if (loading) return Loading...; if (error) return Error: {error};
```

■ Form with Validation

Name + email form, validate before submit

```
const [form, setForm] = useState({ name: "", email: "" }); const [errors, setErrors] =
useState({}); const validate = () => { const errs = {}; if (!form.name) errs.name = "Name
required"; if (!/\S+@\S+\.\S+/.test(form.email)) errs.email = "Invalid email"; return
errs; }; const handleSubmit = () => { const errs = validate(); if
(Object.keys(errs).length) { setErrors(errs); return; } console.log("Submitted", form); };
```

■ Theme Toggle

Light/dark mode toggle with localStorage persist

```
const [dark, setDark] = useState( () => localStorage.getItem("theme") === "dark" ); const
toggle = () => { setDark(prev => { localStorage.setItem("theme", !prev ? "dark" :
"light"); return !prev; }); }; return ( {dark ? "Light" : "Dark"} Mode );
```

■ Accordion

Click to open/close sections, only one open at a time

```
const [open, setOpen] = useState(null); const items = [{id:1, q:"What?", a:"This."},
{id:2, q:"Why?", a:"That."}]; return ( {items.map(item => ( setOpen(open === item.id ?
null : item.id)> {item.q} {open === item.id && {item.a}} ))} );
```

■ Modal / Popup

Button opens modal, X closes it, outside click closes it

```
const [show, setShow] = useState(false); return ( <> setShow(true)>Open {show && (
setShow(false)} style={{position:"fixed",inset:0,background:"rgba(0,0,0,0.5)"}}>
e.stopPropagation()} style={{background:"white",padding:20,margin:"100px
auto",width:300}}> setShow(false)>X Modal Content )) );
```

10 DAY-BEFORE CHECKLIST

Tonight (Before Sleep)

- Say self-introduction 5 times — camera on, record yourself
- Explain ONYX out loud 3 times — use the script from Chapter 8
- Write the component skeleton from memory — no looking
- Write a counter component from scratch — no help, 10 minutes max
- Eat well. Sleep 8 hours. No late night coding.

Interview Morning

- Arrive / join 10 minutes early
- Have ONYX GitHub link ready to share if asked
- Have your resume in hand (printed or PDF open)
- Water bottle. Deep breath before entering.

During the Interview — Mental Rules

- If you don't understand: "Could you repeat that?" — never guess the question
- If you don't know: "I haven't used that directly, but my approach would be..." — never say "I don't know" and stop
- Coding round: skeleton FIRST, always. Speak while you type.
- Bug in your code: "Let me trace through this..." — debugging live is a green flag
- They say "we'll let you know": ask "What is the expected timeline?" — shows professionalism

The One Thing That Will Separate You

Every other candidate knows React basics. What makes you different: you built a system where a LangChain agent decides when to search the web. You solved an OAuth2 refresh token failure in production. You caught a stock overselling race condition and handled it at the checkout endpoint. These are not tutorial outcomes. These are engineering decisions. TALK ABOUT THEM.

Quick Reference Card — Paste This Somewhere Visible

Concept	One Line
Virtual DOM	Lightweight copy; React diffs it to minimize real DOM updates
useState	Component memory — triggers re-render when value changes
useEffect	Side effects after render — API calls, timers, subscriptions
useMemo	Memoize expensive computed value — recalculate only when deps change
useCallback	Memoize function reference — stable reference for child components
Closure	Inner function remembers outer scope even after outer function returns
Event Loop	JS is single-threaded; async callbacks wait in queue for stack to clear
JWT	Signed token: Header.Payload.Signature — server verifies without DB lookup
Middleware	Function between request and response — auth, logging, validation
populate()	Mongoose JOIN — replaces ObjectId with actual document

Router Loader	Fetch data at route level before component renders — no useEffect needed
---------------	--

You have shipped ONYX. You have debugged LangChain in production. You have built a desktop OS in the browser. Walk in like someone who has already done hard things. Because you have.